ECSE 222, Digital Logic

# VHDL 5:

## Storage Elements, Clock Divider, and Counters

Fall 2023
University of McGill

Presented by:

**Nara Yun (Student #: 261115268)**

**Karen Chen Lai (Student #: 261107674)**

Instructors: Prof. Boris Vaisband

Teaching Assistant: Maninder Bir Singh Gulshan

Date: November 25th, 2023

## Executive Summary

In this VHDL Assignment #5, we practiced on the design and implementation of storage elements, a clock divider, and a 3-bit up-counter. This assignment is composed by three main sections: A positive-edge-triggered JK flip-flop with asynchronous active-low reset and set signals; A 3-bit up-counter counting at the positive edge of the clock with an asynchronous reset and an enable signal; And a clock divider generating an enable signal every T clock cycles. The VHDL process blocks were used in the implementation, as well as extensive simulations with ModelSim to ensure the correctness of each circuit. In addition, a 3-bit up-counter was integrated with the clock divider to achieve 1-second interval counting, as demonstrated by a wrapper circuit. Moreover, we executed timing analysis was also required to identify critical paths and delays in the 3-bit up-counter.

## Questions

1. Briefly explain your VHDL code implementation of all circuits.

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;

entity Karen_Chen_jkff_tb is
--empty
end Karen_Chen_jkff_tb;

architecture tb of Karen_Chen_jkff_tb is

   component Karen_Chen_jkff is
      Port (    clk : in std_logic;
                J   : in std_logic;
                K   : in std_logic;
                Q   : out std_logic);

   end component;

   signal clk_in: std_logic;
   signal J_in:  std_logic;
   signal K_in:  std_logic;
   signal Q_out: std_logic;
   --constant clock_period : time := 10 ns ;


   begin
      DUT : Karen_Chen_jkff port map (clk_in, J_in, K_in, Q_out);


      clock_generation : process
   begin
      clk_in <= '1';
      wait for 5 ns;
      clk_in <= '0';
      wait for 5 ns;
   end process clock_generation ;


   process
      begin

         J_in <= '0';
         K_in <= '0';
         wait for 10 ns;

         J_in <= '0';
         K_in <= '1';
         wait for 10 ns;

         J_in <= '1';
         K_in <= '0';
         wait for 10 ns;

         J_in <= '1';
         K_in <= '1';
         wait for 10 ns;

      wait;
   end process;
end tb;
```

**Figure 1.** JKFF Testbench

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Karen_Chen_jkff is
    Port (clk : in std_logic;
          J   : in std_logic;
          K   : in std_logic;
          Q   : out std_logic);
end Karen_Chen_jkff ;

architecture jkff of Karen_Chen_jkff is
   signal input:std_logic_vector(1 downto 0);

begin
   PROCESS (clk)
      variable Qt: std_logic;
      BEGIN


         if RISING_EDGE( clk ) then
            if ( J ='0' AND K = '0') then
               Qt := Qt;
            elsif ( J ='0' AND K = '1') then
               Qt := '0';
            elsif ( J ='1' AND K = '0') then
               Qt := '1';
            elsif ( J ='1' AND K = '1') then
               Qt := NOT (Qt);
            else
               Qt := '1';
            end if;
         end if;
      Q <= Qt;
   end process;
end jkff;
```

**Figure 2.** JKFF behaviorioral description

JK Flip-Flop (JKFF):

The VHDL implementation for the JKFF involves a process block within which the positive edge of the clock is detected using the RISING_EDGE attribute. This ensures that the flip-flop operates on the rising edge of the clock signal, as same as the equation D = JQ' + K'Q. The code includes provisions for asynchronous active-low reset (clear) and set signals, allowing for immediate state changes based on the status of these signals. Importantly, this implementation follows a priority scheme to manage the reset and set conditions, ensuring a robust and predictable behavior.

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;

entity Karen_Chen_counter_tb is
--empty
end Karen_Chen_counter_tb;

architecture tb of Karen_Chen_counter_tb is

    component Karen_Chen_counter is
        Port (   enable   : in std_logic;
                 reset    : in std_logic;
                 clk      : in std_logic;
                 count    : out std_logic_vector(2 downto 0));

    end component;

    signal enable_in: std_logic;
    signal reset_in:  std_logic;
    signal clk_in:  std_logic;
    signal count_out: std_logic_vector(2 downto 0);
    --constant clock_period : time := 10 ns;

    begin

       DUT : Karen_Chen_counter port map (enable_in, reset_in, clk_in, count_out);

       clock_generation : process
            begin
               clk_in <= '1';
               wait for 5 ns;
               clk_in <= '0';
               wait for 5 ns;
            end process clock_generation ;

            process
               begin

                   enable_in <= '0';
                   reset_in <= '0';
                   wait for 10 ns;

                   enable_in <= '0';
                   reset_in <= '1';
                   wait for 10 ns;
                  enable_in <= '1';
                  reset_in <= '1';
                   wait for 10 ns;

               wait;
        end process;
end tb;
```

**Figure 3.** Counter Testbench

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Karen_Chen_counter is
    Port ( enable   : in std_logic;
           reset    : in std_logic;
           clk      : in std_logic;
           count    : out std_logic_vector(2 downto 0));
end Karen_Chen_counter ;

architecture counter of Karen_Chen_counter is

   signal count_signal: integer range 0 to 7;

begin

   process (clk)
      begin

         if (reset = '0') then
            count_signal <= 0;
         elsif RISING_EDGE( clk ) then
            if (enable = '1') then
               if (count_signal = 7) then
                  count_signal<= 0;
               else
                  count_signal <= count_signal + 1;
               end if;
            end if;
         end if;
   end process;
count <= std_logic_vector( to_unsigned(count_signal, 3));

end counter;
```

**Figure 4.** Counter Behaviorioral description

3-bit Up-Counter:

The 3-bit Up-Counter implementation utilizes a process block sensitive to the positive edge of the clock. This block includes logic for asynchronous reset and an enable signal to control the counter's behavior. When the enable signal is active, the counter increments its value, and it wraps around to 0 when it reaches 7 as illustrated in Figure 4.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Karen_Chen_clock_divider is
    Port (    enable   : in std_logic;
              reset    : in std_logic;
              clk      : in std_logic;
              en_out   : out std_logic );
    end Karen_Chen_clock_divider ;

architecture clock_divider of Karen_Chen_clock_divider is

    signal counter :  integer := 49999999;
    -- T = 1/f = 1/ 50M HZ = 20 ns

    begin

        process(reset, clk)

        begin
            if (reset = '0') then
                en_out <= '0';
                counter <=  49999999;
            elsif (RISING_EDGE(clk) and enable = '1') then
                if (counter = 0) then
                    en_out   <= '1';
                    counter  <= 49999999;
                else
                    en_out   <= '0';
                    counter <=  counter - 1;
                end if;
            end if;
        end process;

end clock_divider;
```

**Figure 5.** Clock divider behavior

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;

entity Karen_Chen_clock_divider_tb is
--empty
end Karen_Chen_clock_divider_tb;

architecture tb of Karen_Chen_clock_divider_tb is

    component Karen_Chen_clock_divider is
        Port (    enable   : in std_logic;
                  reset    : in std_logic;
                  clk      : in std_logic;
                  en_out   : out std_logic );
    end component;

    signal enable_in: std_logic;
    signal reset_in:  std_logic;
    signal clk_in:  std_logic;
    signal en_out_out: std_logic;
    -- constant clock_period : time := 100 ms;


    begin
    DUT: Karen_Chen_clock_divider port map(enable_in, reset_in, clk_in, en_out_out);

    clk_generatioin : process
    begin
       clk_in <= '0'; --T = 1/50M HZ = 20 ns
       wait for 10 ns;
       clk_in <= '1';
       wait for 10 ns;
    end process;

    assertion_enable : process
    begin
       enable_in <= '0';
       reset_in <= '0';
       wait for 20 ns;

       enable_in <= '1';
       reset_in <= '1';
       wait for 1000000000 ns;

    end process;

end tb;
```

**Figure 6.** Clock divider Testbench

Clock Divider:

The Clock Divider is implemented using a down-counter within a process block. The counter counts down from a T-1 to 0, and the enable signal is asserted when the count reaches 0. While the output of the clock divider is 1 and the count will reset to T-1 when reaching the count of 0. Otherwise, the output of the clock divider is 0. In this laboratory, the frequency is 50MHz, in which the clock period is 20 ns as shown in Figure 6, testbench code for clock divider.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Karen_Chen_wrapper is
    Port (enable : in std_logic;
          reset : in std_logic;
          clk : in std_logic;
          HEX0 : out std_logic_vector (6 downto 0));
end Karen_Chen_wrapper;

architecture wrapper of Karen_Chen_wrapper is

    component Karen_Chen_clock_divider is
        port (    enable   : in std_logic;
                  reset    : in std_logic;
                  clk      : in std_logic;
                  en_out   : out std_logic );
    end component;

    component Karen_Chen_counter is
        port ( enable   : in std_logic;
               reset    : in std_logic;
               clk      : in std_logic;
               count    : out std_logic_vector(2 downto 0));
    end component;

    component seven_segment_decoder is
        port(code: in std_logic_vector (3 downto 0);
            segments_out: out std_logic_vector (6 downto 0));
    end component;

    signal en_out_out : std_logic;
    signal count_out : std_logic_vector(2 downto 0);
    signal code1 : std_logic_vector (3 downto 0);

    begin

        clock: Karen_Chen_clock_divider port map (enable, reset, clk, en_out_out);

        counter: Karen_Chen_counter port map (en_out_out, reset, clk, count_out);

        code1 <= ("0" & count_out);

        decoder: seven_segment_decoder port map (code1, HEX0);

end wrapper;
```

**Figure 7.** 3-bit up counter Wrapper behavioral description

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;

entity Karen_Chen_wrapper_tb is
--empty
end Karen_Chen_wrapper_tb;

architecture tb of Karen_Chen_wrapper_tb is

component Karen_Chen_wrapper is
   Port    ( enable  : in std_logic ;
             reset   : in std_logic ;
             clk     : in std_logic ;
             HEX0    : out std_logic_vector(6 downto 0));
end component;


   signal enable_in: std_logic;
   signal reset_in:  std_logic;
   signal clk_in:  std_logic;
   signal HEX0_out: std_logic_vector(6 downto 0);
   --constant clock_period : time := 100 ms;

   begin

   DUT : Karen_Chen_wrapper port map (enable_in, reset_in, clk_in, HEX0_out);

   clock_generation : process
   begin
      clk_in <= '1';
      wait for 10 ns;
      clk_in <= '0';
      wait for 10 ns;
   end process clock_generation ;

  process
   begin

   enable_in <= '0';
   reset_in <= '0';
   wait for 10 ns;

   enable_in <= '0';
   reset_in <= '1';
   wait for 10 ns;

   enable_in <= '1';
   reset_in <= '0';
   wait for 10 ns;

   enable_in <= '1';
   reset_in <= '1';
   wait for 10 ns;

    wait;
  end process;
end tb;
```

**Figure 8.** 3-bit up counter Wrapper testbench code

3 bit up-counter counting in increments of 1 second:

Both Fig. 7 and Fig. 8 are the the descriptive codes for the 3-bit up counter in increments of 1 seconds. In the Figure 7, it utilizes the clock divider, counter and the seven segment decoder to count every 1 second that contains 50 M times of risings in the clock divider. Then it transfers the counted values into decoded values in seven segment decoder.

2. Provide waveforms for each of the individual storage elements and sequential circuits (each section) and for the 3-bit up-counter.

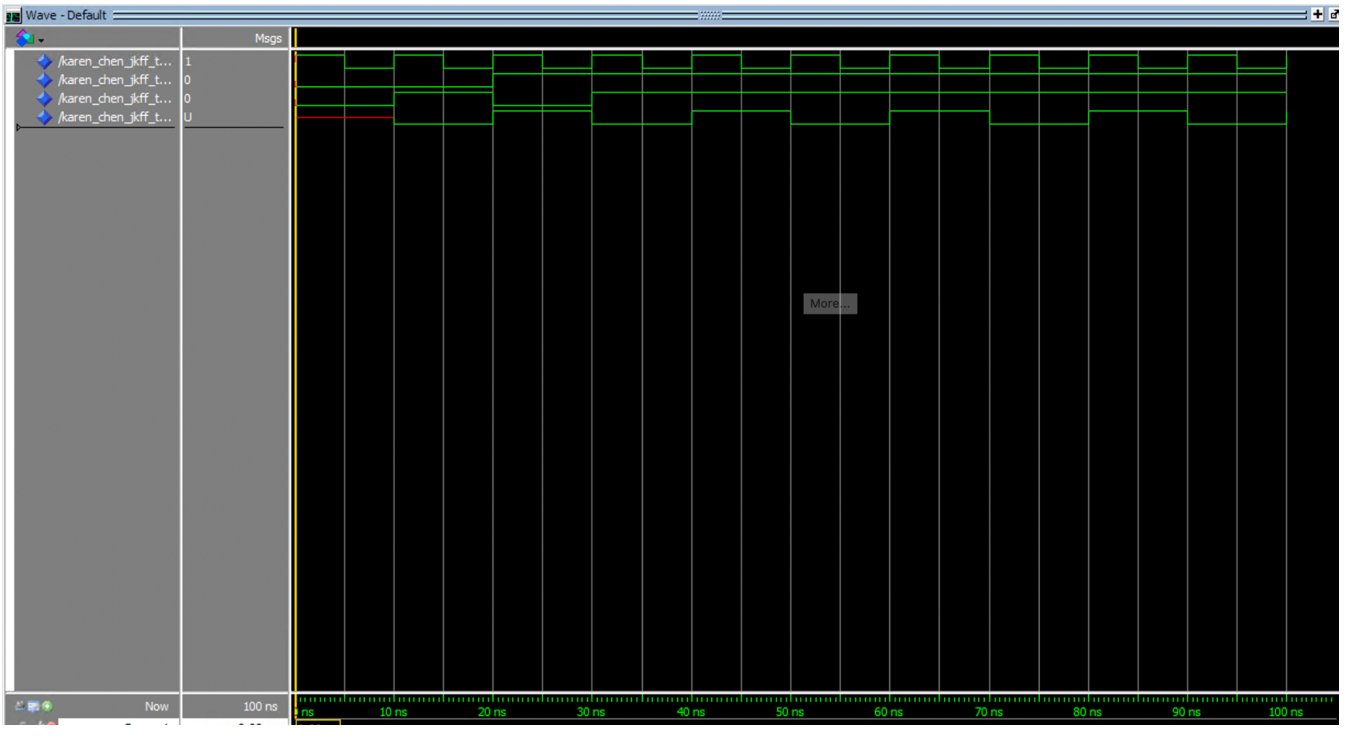

**Figure 8.** Simulation plot of JKFF

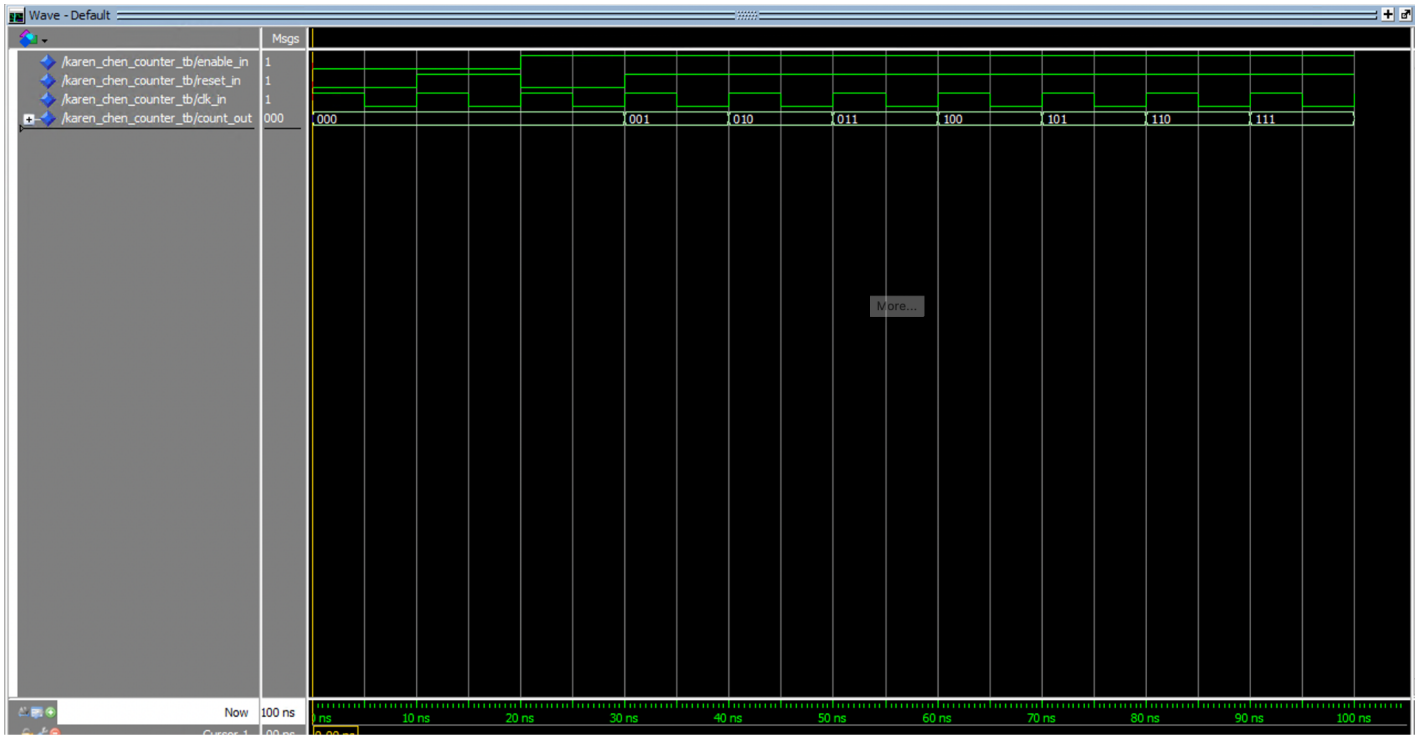


**Figure 9.** Simulation plot of counters

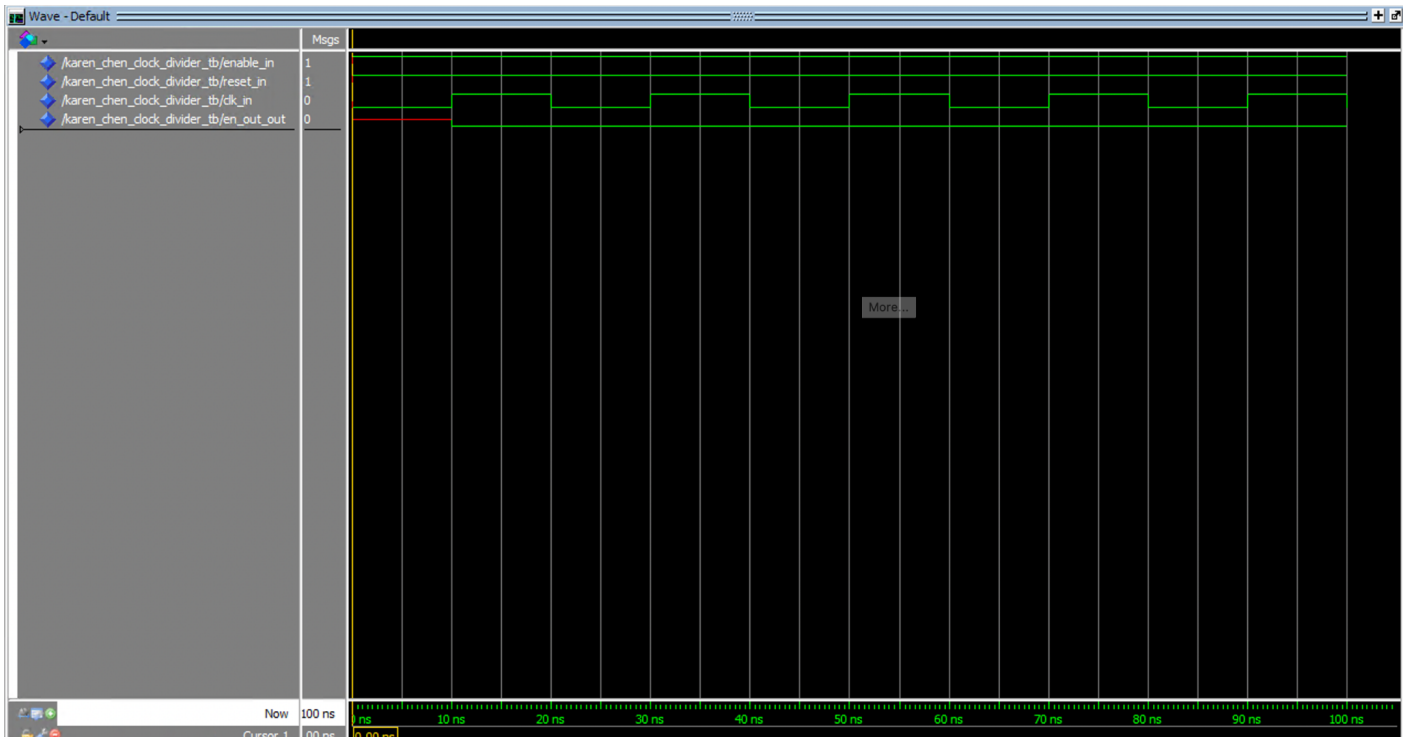


**Figure 10.** Simulation plot of clock divider

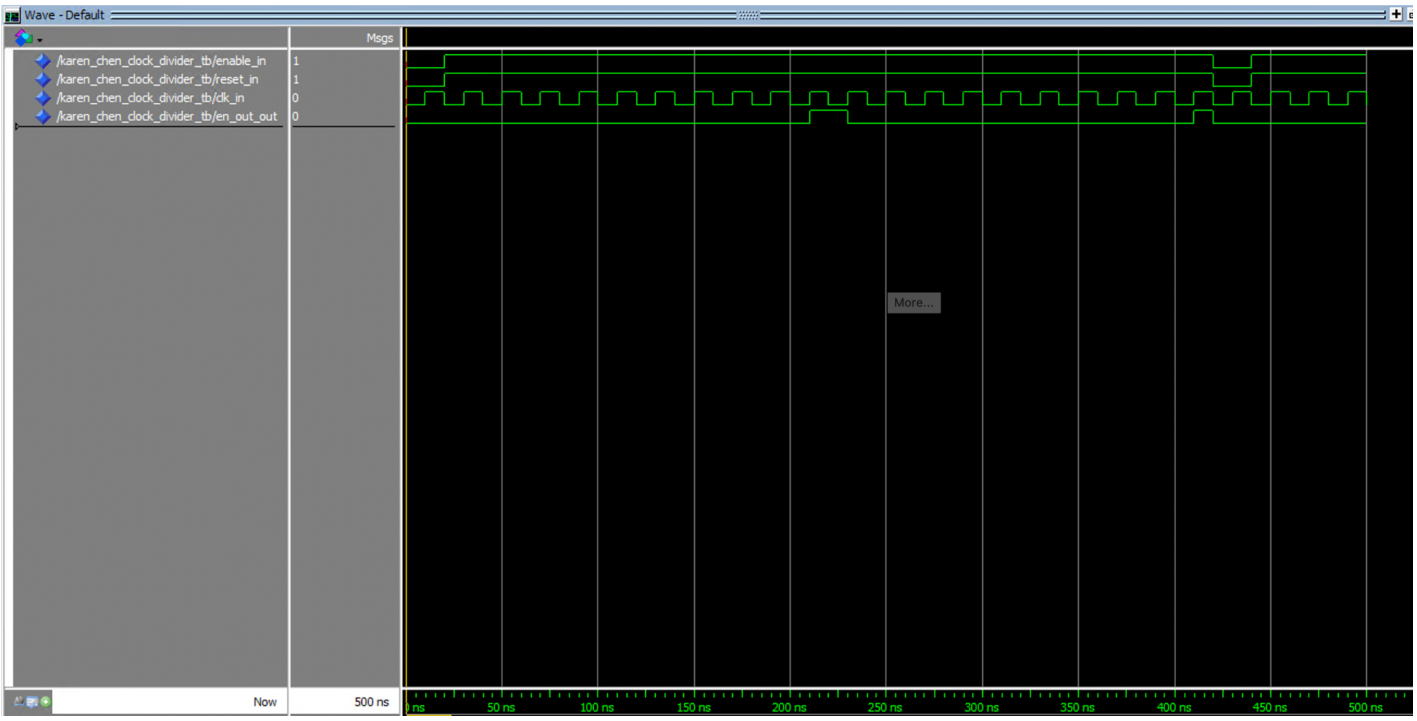


**Figure 11.** Simulation plot of TESTING clock divider with a frequency of 10 HZ

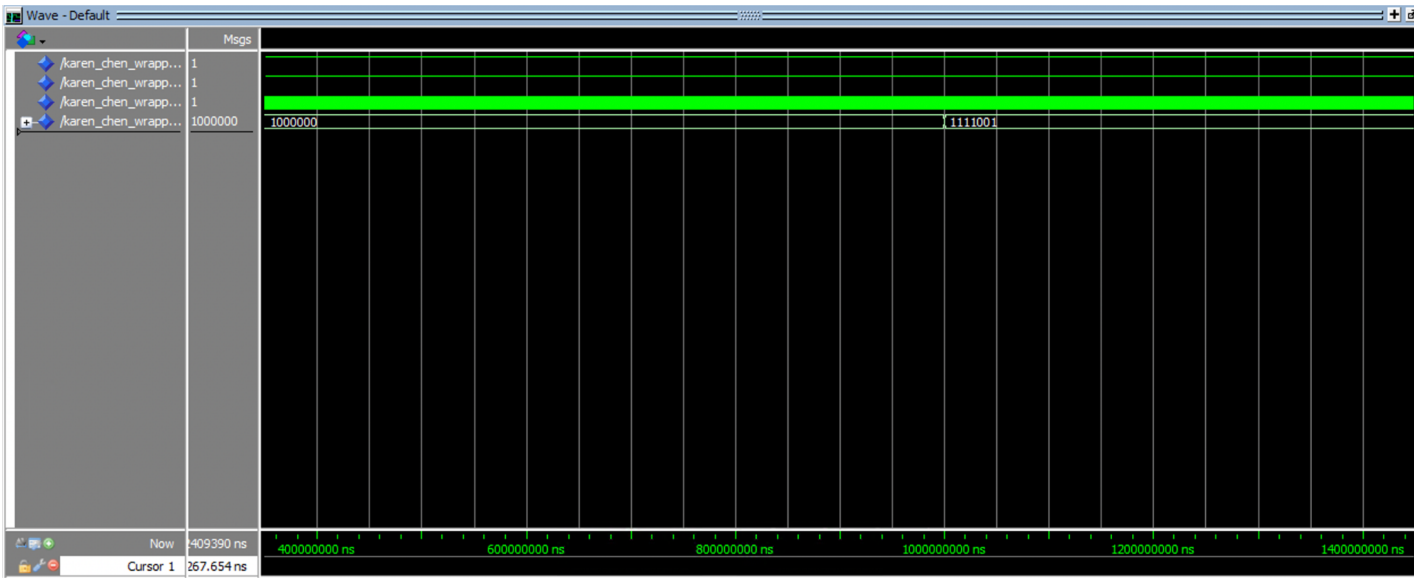


**Figure 12.** Waveform of Wrapper after 200 000 000 ns

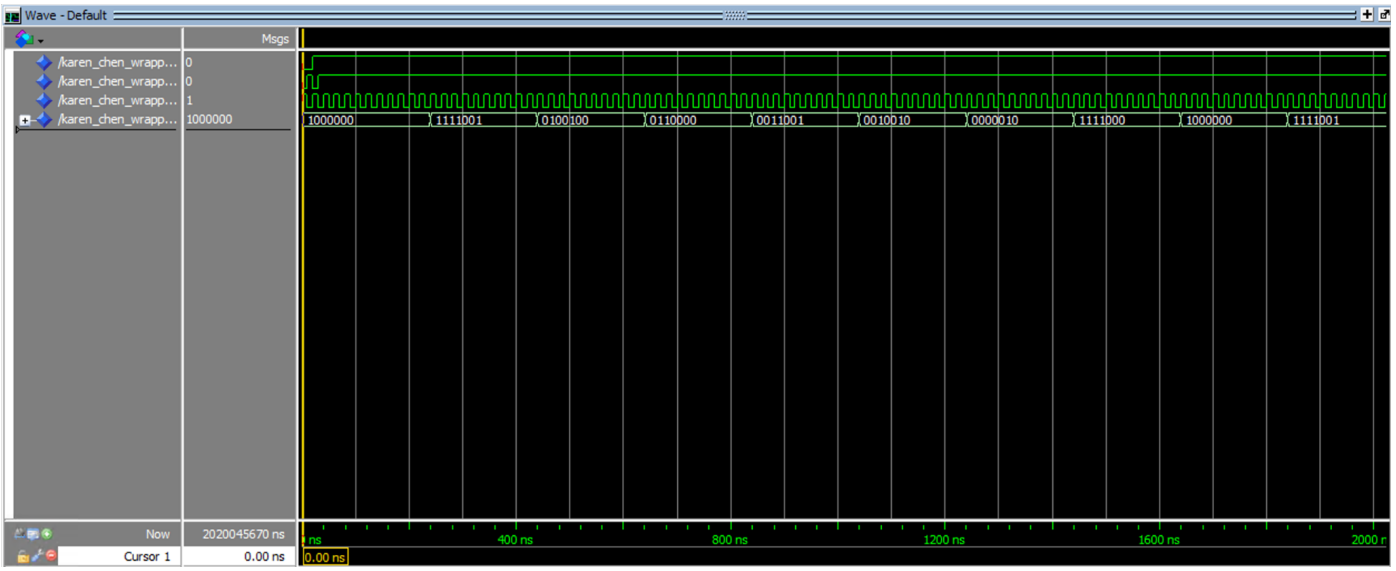


**Figure 13.** Waveform of TESTING wrapper with a frequency of 10 HZ

3. Perform timing analysis (slow 1,100 mV, 85C model) of the 3-bit up counter and find the critical path(s) of the circuit. What is the delay of the critical path(s)?

**Top Failing Paths [hide details]**

| | Slack | From | To | Recommendations |
|---|---|---|---|---|
| 1 | -2.855 | Karen_Chen_clock_...clock\|counter[6] | Karen_Chen_clock_...lock\|counter[30] | Report recommendations for this path |
| 2 | -2.841 | Karen_Chen_clock_...clock\|counter[6] | Karen_Chen_clock_...lock\|counter[21] | Report recommendations for this path |
| 3 | -2.837 | Karen_Chen_clock_...clock\|counter[6] | Karen_Chen_clock_...lock\|counter[20] | Report recommendations for this path |
| 4 | -2.813 | Karen_Chen_clock_...clock\|counter[6] | Karen_Chen_clock_...lock\|counter[25] | Report recommendations for this path |
| 5 | -2.744 | Karen_Chen_clock_...lock\|counter[11] | Karen_Chen_clock_...lock\|counter[30] | Report recommendations for this path |

**Figure 14.** Top critical paths

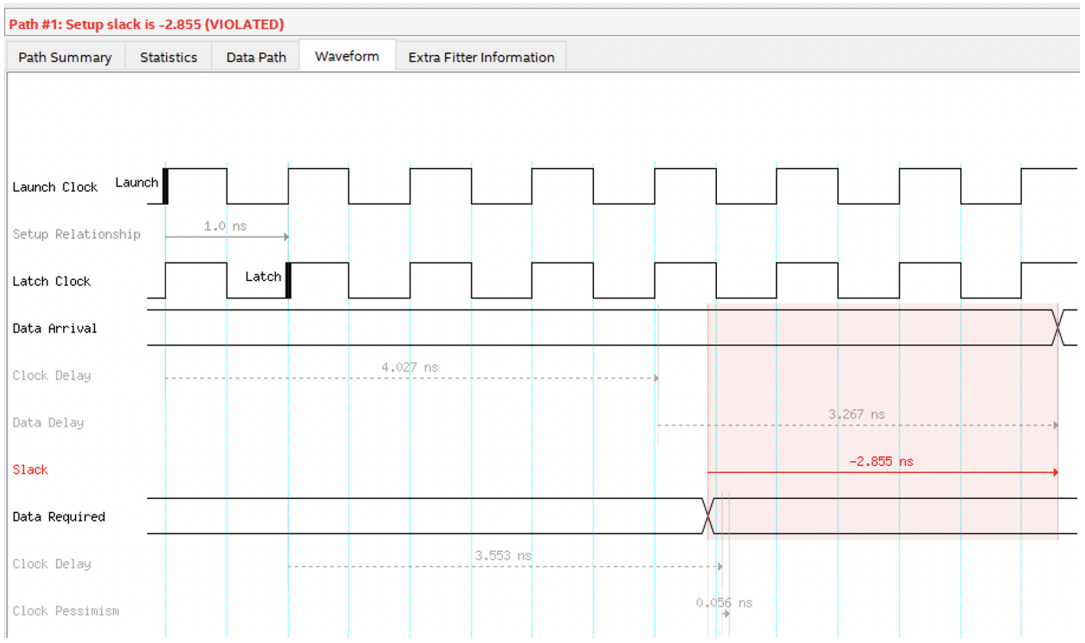


**Figure 15.** Waveform of the critical path

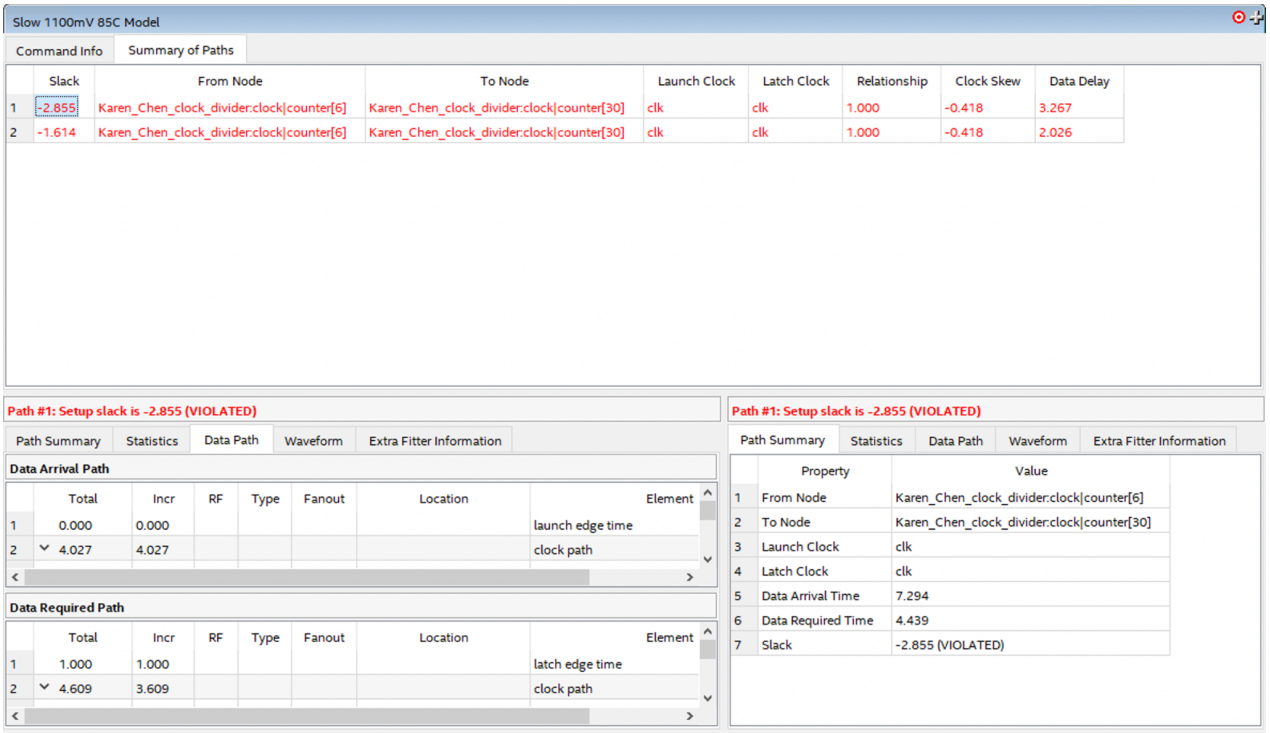


**Figure 16.** Timing analysis of critical path of the circuit under Slow 1100mV 85C Model

As Figure 16 shows, the critical path has a slack of 2.855 and a data delay of 3.267.

4. Report the number of pins and logic modules used to fit your 3-bit up counter design on the FPGA board.

Number of pins: 10/457 (2%)

Number of logic Modules: 40/ 32 070 (< 1%)

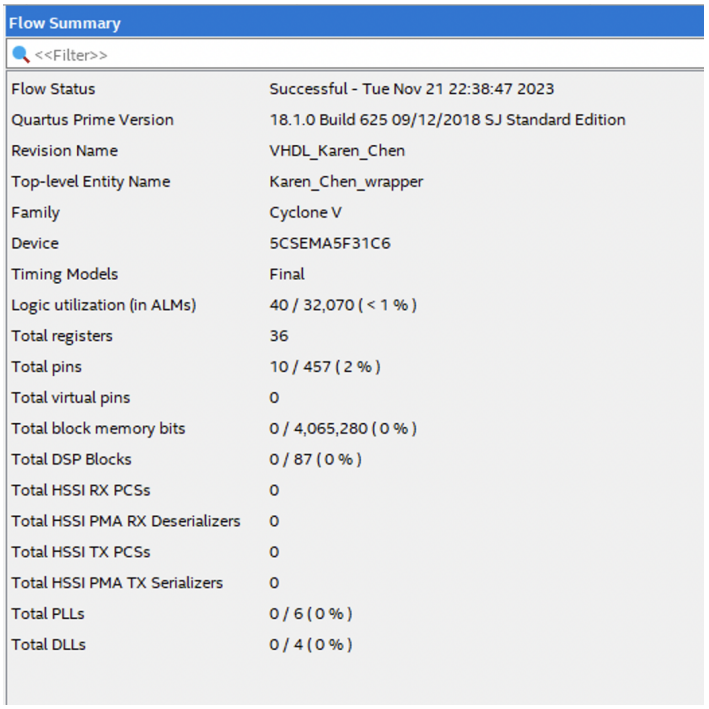


**Figure 17.** Flow summary of 3-bit up counter in increment of 1 second

**Explanation of the Results**

Fig 8. performs the JKFF simulation. In which operates like the boolean function D = JQ' + K'Q. When both J and K are 0, the the input will maintain state; when the J is 0 and K is 1, the input will be reset to 0; when the J is 1 and K is 0, the input will be set to 1; and when the both J and K are 1, the inout will be inversed.

Fig. 9 illustrates the performance of 3 bit up counter with an asynchronous reset and an enable signal whic is always '1'. The counter will counts up when the enable signal is high at the positive edge of clock. It will counts from 0 until 7, it will reset to 0 when it reaches 7.

Fig. 10 is the waveform of the clock divider, in which does not show up the result very perfect because we set up the running time range to 100 ns and this is not long enough to expect a change in the output when it finishes going through every T clock cycle. Therefore, we tested our clock divider with a smaller frequency, 10HZ, as illustrated in Fig. 11. It demonstrates that every 10 times of clock rising, the output turns to 1 when both the enable is 1 and reset is 1.

Fig. 12 is the RTL simulation plot of 3-bit up-counter in increments of 1 second. The output counts up every 1 second, in other words, after 50 000 000 times of clock rises, the count rises up 1. Where will also transfer the output of the counter to the seven segment decoder in order to produce a decoded value. Similar to the clock divider, we also tested this 3 bit up-counter with a frequency of 10 HZ in Fig. 13, where it shows clearly that the circuit outputs every segment in decoded value in increasing order. And when it reaches 7, it will turn to 0 in decoded value to recount again.

**Conclusions**

Through this experiment, we learned how to:

- Implement a positive-edge triggered JK Flip-Flop with asynchronous active-low reset and set signals using VHDL process blocks.
- Design an asynchronous 3-bit Up-Counter when the positive clock rise happens with both reset and enable signals.
- Create a Clock Divider that outputs an enable signal every T clock cycles for accurate timing.
- Design a wrapper circuit using clock divider counter and seven segment decoder to achieve counting at 1-second intervals.
- Practiced and enhanced our comprehensive understanding on importance of timing delays and its effect through real-time operation on ModelSim-Altera.